



Módulo 14: Proyecto Final Integrador - Desarrollo Colaborativo en GitHub (CSIC- ClimateWatch)

- **Curso:** Python para Ciencia Abierta (CSIC)
- **Autor:** Gustavo Liñán Cembrano
- **Fechas:** 5 a 8 de Octubre de 2026
- **Tiempo Estimado de Impartición:** 2 horas
- **Módulo:** 14 (Proyecto Final de Integración y Trabajo Colaborativo en GitHub)
- **Repositorio Oficial Base:**
https://github.com/guslicem/proyecto_final_curso_python_2026

 **Objetivos Didácticos de la Práctica Integradora**

1. **Estructura de Software Científico FAIR:** Comprender la organización modular de una librería profesional en Python organizando código backend (`src/`), interfaz interactiva (`app.py`), pruebas unitarias (`tests/`) y automatización CI/CD (`.github/workflows/ci.yml`).
2. **Dinámica de Investigación Colaborativa:** Simular un entorno real de desarrollo abierto dividiendo el grupo en 4 equipos de trabajo especializados con tareas accesibles para programadores no expertos (~45 min).
3. **Flujo de Trabajo Git Completo:** Ejecutar el ciclo de *Fork*, clonado local, creación de ramas descriptivas (`feature/...`), *commits* atómicos, sincronización remota (*push*), apertura de *Pull Requests* (PRs) y revisión de código por pares (*Peer Review*).
4. **Despliegue y Publicación de Release Oficial (`v1.0.0`):** Integrar todas las aportaciones de los equipos y etiquetar la primera versión pública del proyecto.



Demostración del Estado Actual de la Aplicación Streamlit

Vamos a ver la app **CSIC-ClimateWatch** desplegada en vivo para mostrar el producto final que váis a evolucionar:

```
# 1. Abrir la terminal de Antigravity-IDE o sistema y navegar al  
proyecto final  
cd PROYECTO_FINAL
```

```
# 2. Asegurarse de tener activado el entorno virtual  
source ../venv/bin/activate
```

```
# 3. Lanzar el cuadro de mando interactivo en Streamlit  
streamlit run app.py
```

1. Presentación del Proyecto CSIC-ClimateWatch

He creado este proyecto **CSIC-ClimateWatch** que es una aplicación interactiva y librería modular en Python diseñada para auditar el Cambio Climático, las anomalías térmicas y la sequía en España a partir de observaciones meteorológicas reales del reanálisis **ERA5** (Copernicus / ECMWF) y la metodología del **Índice SPEI** desarrollada por el **Instituto Pirenaico de Ecología (IPE-CSIC)** ([Digital.CSIC Handle: 10261/22405](https://digital.csic.es/handle/10261/22405)).

Estructura Modular del Repositorio

```

proyecto_final_curso_python_2026/
├── .github/
│   └── workflows/
│       └── ci.yml # Integración continua con GitHub
├── Actions
├── .streamlit/
│   └── config.toml # Configuración de tema oscuro
├── nativo
├── assets/
│   └── logoCSIC.jpg # Logo oficial para el encabezado
├── web
├── data/
│   └── csic_climate_spain.csv # Dataset climático (13.056
observaciones reales 1961–2024)
├── src/
│   └── csic_climate/
│       ├── __init__.py
│       └── data_loader.py # Módulo de carga y filtrado de
datos
│   └── metrics.py # Módulo de cálculo de
indicadores climáticos
├── tests/
│   ├── __init__.py
│   └── test_metrics.py # Arnés de pruebas unitarias
├── pytest para métricas
│   └── test_data_loader.py # Pruebas unitarias para el
cargador de datos
├── app.py # Interfaz gráfica interactiva en
Streamlit
├── pyproject.toml # Configuración de empaquetado
├── pyOpenSci / Hatch
├── requirements.txt # Dependencias del proyecto
├── CITATION.cff # Archivo FAIR de citación del
software
├── CHANGELOG.md # Historial de cambios del
proyecto
├── README.md # Documentación principal del
repositorio

```

2. Plan de Acción por Equipos de Trabajo (~45 min)

La clase se dividirá en 4 grupos especializados con tareas concisas y de alto impacto:

GRUPO 1: Backend & Ciencia de Datos (src/csic_climate/metrics.py)

- **Tarea:** Implementar una nueva función analítica en `metrics.py`:

```
def get_hottest_and_coldest_year(  
    df: pd.DataFrame,  
    start_year: int = 1961,  
    end_year: int = 2024,  
    comunidades: Union[List[str], str] = "all"  
) -> Dict[str, Any]:
```

- **Requisitos:**

1. Filtrar los datos por el rango `[start_year, end_year]`.
 2. Si `comunidades` es una lista (ej. `["Andalucía", "Aragón"]`), filtrar solo esas regiones; si es `"all"`, incluir todas.
 3. Devolver un diccionario con el año más cálido y el año más frío (según la anomalía térmica media anual) y sus respectivos valores.
-

GRUPO 2: Frontend & Streamlit (`app.py`)

- **Tarea:** Mejorar la visualización sobre días de ola de calor en la interfaz de Streamlit.
 - **Requisitos:**
 1. Calcular el número acumulado de días en ola de calor (`dias_ola_calor`) en el periodo y región seleccionados.
 2. **Alerta Dinámica:** Si el número total de días supera los **20 días**, mostrar un aviso destacado con `st.warning("⚠️ Alerta Climática: Se han superado los 20 días en ola de calor en el periodo seleccionado")` .
 3. **Comparativa:** Mostrar la diferencia en días frente al promedio de todas las comunidades en ese mismo rango temporal.
-

GRUPO 3: Documentación & Ciencia Abierta (`README.md` , `CITATION.cff` y `CHANGELOG.md`)

- **Tarea:** Completar la documentación FAIR y mantener el registro de cambios.
 - **Requisitos:**
 1. **Glosario en `README.md`** : Redactar la tabla descriptiva con el significado científico y unidades de las 9 columnas del CSV.
 2. **Citación:** Completar los metadatos de autoría y URL en `CITATION.cff` y el bloque BibTeX.
 3. **Control de Cambios:** Editar `CHANGELOG.md` registrando los nuevos desarrollos a medida que los Equipos 1, 2 y 4 habran sus Pull Requests.
-

GRUPO 4: Testing & Calidad de Software (tests/)

- **Tarea:** Extender el arnés de pruebas con `pytest` .
- **Requisitos:**
 1. Actualizar `tests/test_metrics.py` añadiendo un test unitario para la nueva función `get_hottest_and_coldest_year` .
 2. Crear el nuevo archivo `tests/test_data_loader.py` con pruebas para `load_climate_data()` (verificar que el DataFrame se carga correctamente y convierte `fecha` a datetime) y `filter_by_region_and_years()` (probar filtrado por comunidad única, "Todas" , y rangos personalizados).

3. Flujo Paso a Paso de Git y GitHub

Paso 1: Hacer un Fork del Repositorio Base

Cada integrante o representante debe acceder a https://github.com/guslicem/proyecto_final_curso_python_2026 y hacer clic en **Fork** (esquina superior derecha).

Paso 2: Clonar el Repositorio en Local

Abre la terminal y clona tu copia personal:

```
git clone
https://github.com/<tu_usuario>/proyecto_final_curso_python_2026.g
it
cd proyecto_final_curso_python_2026
```

Paso 3: Crear una Rama (*Feature Branch*)

NUNCA realices cambios directamente en `main` . Crea una rama con el nombre de tu equipo:

Ejemplo Equipo 1

```
git checkout -b feature/backend-extreme-years
```

Ejemplo Equipo 2

```
git checkout -b feature/frontend-heatwave-alert
```

Ejemplo Equipo 3

```
git checkout -b feature/docs-fair-changelog
```

Ejemplo Equipo 4

```
git checkout -b feature/testing-dataloader-suite
```

Paso 4: Probar los Cambios en Local

Valida que las pruebas unitarias y la aplicación web funcionen correctamente:

Ejecuta el entorno virtual, a estas alturas del curso no debes necesitar que te diga cómo.

```
print("Lo siento se acabo mi ayuda")
```

Ejecutar suite de pruebas pytest

```
pytest
```

Probar la aplicación en Streamlit

```
streamlit run app.py
```

Paso 5: Confirmar Cambios (*Commits*) y Subir (*Push*)

```
git add .
```

```
git commit -m "feat(<equipo>): DESCRIPCION BREVE DE LO QUE HAS CAMBIADO. SE ESCUETO!!!"
```

```
git push -u origin feature/<nombre_rama>
```

Paso 6: Abrir Pull Request (PR) y Revisión por Pares (*Peer Review*)

1. Entra a tu repositorio en GitHub y haz clic en **"Compare & pull request"**.
2. Redacta un título claro y una breve descripción del cambio.
3. Solicita la revisión (*Reviewer*) de un compañero de otro equipo.

4. Integración Directa y Publicación de la Release v1.0.0

Una vez que los 4 equipos hayan abierto sus Pull Requests y hayan sido revisadas por sus compañeros:

1. Revisión y Merge (Gustavo Liñán Cembrano):

- Se revisan las 4 PRs en vivo en la pantalla del aula.
- Se comprueba que las pruebas automatizadas de GitHub Actions (`ci.yml`) estén en verde .
- Se efectúa el *Merge* de cada PR a la rama `main`.

2. Etiquetado y Publicación de la Release Oficial:

- En la consola local:

```
git checkout main
git pull origin main
git tag -a v1.0.0 -m "Release v1.0.0 – Versión Final
Integrada por los Alumnos del CSIC"
git push origin v1.0.0
```
- En GitHub: ir a **Releases** -> **Draft a new release** -> Seleccionar etiqueta `v1.0.0` -> **Publish Release**.

5. Conclusiones del Curso de Python para Ciencia Abierta

¡Enhorabuena! Has completado con éxito la 2ª Edición del Curso *Python para Ciencia Abierta (CSIC)*.

Competencias Adquiridas:

1. **Programación Científica:** Dominio de datos estructurados en Python, Pandas y Matplotlib.
2. **Entornos de Desarrollo Modernos:** Configuración de Antigravity-IDE, extensiones Pylance/Jupyter y asistentes de IA.
3. **Curación y Datos FAIR:** Interacción con Digital.CSIC mediante OAI-PMH, licencias libres y citación de software.
4. **Ingeniería y Colaboración:** Desarrollo modular, pruebas unitarias automatizadas (`pytest`), cuadros de mando (`Streamlit`) y control de versiones profesional en GitHub.

Recursos de Interés

- **Repositorio Base de Todo el Curso**https://github.com/guslicem/PYTHON_PARA_CIENCIA_ABIERTA_2
- **Repositorio Base Proyecto Final:**
https://github.com/guslicem/proyecto_final_curso_python_2026

- **Digital.CSIC:** <https://digital.csic.es/>
- **Estándares pyOpenSci:** <https://www.pyopensci.org/>



Cuestionario de Auto-evaluación y Participación

¡Pon a prueba los conocimientos adquiridos en este módulo! Completa este breve cuestionario interactivo de 5 minutos en Google Forms:



Cuestionario Módulo 14 en Google Forms